

InstantLink & FlowOne: Clear Notes on Provisioning, Activation, Order Management, and Related Concepts

Provisioning and Activation

Provisioning and Activation adds a new subscriber to the network and activates ordered services. It functions independently or as a phase in Service Order Delivery.

How Provisioning and Activation Works

- A customer-facing system (e.g., CRM) sends a request via the Business Support System (BSS) interface.
 - InstantLink validates the request and passes it to Business Service Tool (BST), which processes it in a dedicated workflow.
 - Workflow queries Catalog for decomposition of the ordered Customer Facing Service (CFS).
 - Catalog returns an ordered list of tasks.
 - Workflow executes tasks sequentially in Element Management Systems (EMS) and Network Elements (NE) to activate Resource Facing Services (RFS) of the CFS.
 - Examples of tasks:
 - o Create subscription in HSS.
 - o Add Access Point Name (APN) for 4G in HSS.
 - o Set Quality of Service (QoS) in Policy Control and Rating Function (PCRF).
 - Tasks are sent to the network via Network Element Interfaces (NEI), which translate parameters into the required format for the receiving system.
 - After all tasks complete, workflow builds a response with the overall outcome and any return parameters, sending it back to the originating system.
 - Subscriber can then use the purchased services.
-

InstantLink's 6 Main Components

Each component plays a key role in request processing.

1. BSS Interface

- Links request-originating systems to InstantLink.
- Supports various northbound/customer-facing systems (e.g., sales order system, CRM, OrderHub) to send requests and receive responses.

2. Request Processing Engine (RPE)

- Validates new requests.
- Queues them for processing.
- Organizes and sends responses back to BSS.

3. Service Module Engine (SME)

- Receives requests from RPE.
- Processes them into tasks.
- Sends responses to RPE.
- BST runs inside SME.

4. Task Engine (TE)

- Receives tasks from SME.
- Sends tasks via NEI to external/southbound systems.
- Sends task responses back to SME.

5. Network Model Manager (NEMO)

- Stores information about connected NE or external systems.
- Informs TE which NEI to use for a task.
- Tracks availability and status of external systems.

6. Network Element Interface (NEI)

- Connects TE to specified external systems.
 - Translates task parameters into the required format for the external system.
-

Service Order Delivery

More complex than basic provisioning/activation, involving scheduling and multiple phases.

- Starts when a customer-facing system (e.g., sales order system) sends a fulfillment order via BSS interface.
 - Order passes to BST and is assigned a workflow.
 - Workflow retrieves decomposition from Catalog, which returns activities and tasks.
 - Workflow creates a schedule of activities in Order Management (OM).
 - Activities are grouped into phases: Reserve, Provision, Install, Commit.
 - Users can view order status in FlowOne to ensure on-time delivery.
 - Workflow executes tasks per Catalog order:
 - o Reserve phase: Reserve network resources from inventory; reserve IP address/subnet.
 - o Provision phase: Provision subscriber/services to network.
 - o Install phase: Manual task to OrderHub (e.g., engineer installs CPE like modem).
 - o Commit phase: Create customer/services in UIV; assign inventory; add to billing system.
 - Tasks can be added via workflow or library workflows defined in Catalog.
 - InstantLink updates order status in OM after each activity for real-time visibility.
 - On completion, workflow sends response to originating system.
-

The Architecture

Divided into three parts: BSS Interface, Request Processing, Task Processing.

- CRM sends request (e.g., via InstantLink SOA Web Services BSS Interface).
- RPE validates, accepts, and queues the request.
- RPE passes request to SME, entering BST workflow.
- BST retrieves service decomposition from Catalog cache.
- BST generates tasks in decomposition order.
- SME sends each task to TE, which queues it by NE.
- TE gets NEI details from NEMO and sends task to relevant NEI.
- NEI translates/executes task in external system; external system responds with outcome.
- NEI sends response to TE, which forwards to BST.
- Process repeats until all tasks executed.

- BST creates summary response; sends via SME to RPE.
 - RPE queues response and sends back to CRM via BSS Interface.
-

Request Processing

In Provisioning and Activation, requests fall into two categories: Standard/Lite (produce tasks) and Orders (time-scheduled with activities).

Types of Requests

- **Standard Request:** Normal provisioning/activation; tracked; produces one or more tasks.
- **Lite Request:** Time-critical; not stored in DB (untrackable); produces tasks (e.g., query subscriber profile).
- **Order:** Standard request with schedule; contains activities; each activity produces at least one task.

Request Details

- Originates from BSS; defines subscriber, products/services, high-level action (e.g., create/display).
- In Catalog-driven mode, product/service used to get decomposition from Catalog.

Task Details

- Generated from request via workflow or Catalog decomposition.
 - Defines target external system and operation.
-

Request Priority Management

- Incoming requests get priority from sending system or InstantLink default.
- RPE sets priority based on system-wide default or client user value.
- Requests queued by priority and arrival time.
- If start date specified, request waits until that time before queuing.
- Priority 1 jumps to queue top.

Request Statuses

Status	Description
Initiated	Request received and started

Status	Description
	processing.
Timed	Waiting for scheduled start time.
Started	Processing begun.
Pending	Awaiting external response or condition.
Canceled	Request aborted.
Rejected	Invalid or failed validation.
Ready	Prepared for execution.
Partly Failed	Some tasks failed, but overall partial success.
Failed	Complete failure.

Task Statuses

Status	Description
Initiated	Task created.
Queued at RPE	In RPE queue.
Queued at TE	In TE queue.
Pending	Awaiting execution.
Canceled	Task aborted.
Rejected	Invalid task.
Ready	Prepared for execution.
Partly Failed	Partial task failure.
Failed	Complete failure.
Returned	Response sent back.
Executing	In progress.
Internal	Internal processing.
Timed	Scheduled wait.

What is Order Management?

Order Management (OM) provides visibility and control for complex service delivery.

Why Need an Order Management System?

- Operators deliver complex services (e.g., ADSL with CPE, manual cross-connections).
- Requires constant order visibility/control.
- Consumers demand anytime fulfillment via multiple channels and progress details.

Challenges in Operator Business

- **Agility/Time-to-Market:**
 - o Processes in silos.
 - o Hosted across systems/vendors.
 - o Unclear sales/care/delivery division; spaghetti architecture.
 - o No SOA transactions for sales/care apps.
 - o No product/service management; data hardcoded/fragmented.
- **Day-to-Day Operations:**
 - o Lack of order visibility due to fragmentation.
 - o Difficult inflight order control due to architecture.

Key Takeaways

- Order: Work order from BSS.
 - Consists of activities.
 - Baseline: Pre-configured processing estimation.
 - Current Schedule: Real-time estimation.
 - Activities can process in parallel or sequence.
-

Jeopardy Management

What is Jeopardy?

- Risk that order/activity won't complete by required date.
- Order in jeopardy if completion after customer required date.
- Activity in jeopardy if exceeds assigned duration.

What is Point of No Return?

- Milestone (often in manual activity, e.g., delivering modem) after which cancellation requires non-OM operations.

- Example: Break in schedule (e.g., weekend wait); if not started, order cancelable.
 - Post-point: Handle via delete subscription/new order (e.g., after dispatching physical goods).
-

Order Management Inflight Operations

Three types: Reschedule, Cancel, Amend.

1. Reschedule

- Changes customer required date; affects jeopardy status.
- Example: Customer requests later finish.

2. Cancel

- Order no longer valid.
- Workflow handles: Rollback completed activities; send notifications.

3. Amend

- Changes to active order, e.g.:
 - New parameter values.
 - Insert new parameters.
 - Change installation date for CPE.
-

Date Change Layer

- Front-end staff creates rescheduling order (same order number/ID).
- Sent to OM; schedule updated.

Cancel Layer

- Performed via OM web services or UI button.
- Rollback process defined separately to recover completed activities.

Amend

- Amend order with new parameters/values sent to OM.
- Details updated per configured workflow.

Summary (Jeopardy & Inflight Operations)

- Jeopardy: Risk of late completion; status colors: Yellow (started), Orange (canceled/completed).
 - Reschedule: From UI/BSS; updates activities' schedules in OM workflow.
 - Both orders/activities reschedulable.
 - Amend: From BSS only; cancels/re-submits pending activities with new values; may add notification/confirmation activities.
 - Cancel: From UI/BSS; workflow-controlled; cancels existing activities; creates rollback activities separately.
 - All inflight operations defined/controlled by workflow.
 - Amend details in OM's task parameters section.
-

Key Components of FlowOne Catalog

Overview

- Telecom competition drives complex products (e.g., blended access, apps, content).
- Catalog manages product lifecycle centrally; breaks offerings into reusable elements.
- Develops/accesses product/service specs from one place; built from reusable objects (e.g., catalog products).

Functionality

- Uses generic "items" concept for entities; defines item types for encapsulation (e.g., Product Specification > CFS Specs > RFS Specs > Resources).
 - Streamlines lifecycle: Retire/upgrade products efficiently.
 - Interfaces:
 - o Catalog API: SOAP for external create/retrieve.
 - o BST Interface: Internal for Provisioning/Activation.
 - o NEI: Via InstantLink SDK for transactions on catalog items.
 - Security: Username/password auth; role-based authorization by item type; user groups restrict access.
-

Catalog in FlowOne Fulfilment

Components

- OrderHub (top layer).
- FlowOne: OM, Catalog, P&A, UIV.
- Network and Infrastructure (bottom layer).

FlowOne Overview

- Solution for creating/delivering telecom services across multi-vendor networks.
- Uses Catalog-Driven Fulfillment (CDFF): Network-agnostic for designing/executing/managing end-to-end processes.
- Leverages pre-configured/reusable assets for converged services.
- Automates via OM, provisioning, activation.
- Catalog defines service structures; Unified Inventory tracks subscriber/services/resources.
- OrderHub: Configurable interface for order delivery, entry, fallout.

Order Fulfillment Steps (e.g., Triple Play Connect)

1. CRM sends order to OM northbound API; OM records and acknowledges.
 2. OM starts CDFF workflow; queries Catalog for decomposition; creates delivery plan from activities.
 3. OM reserves resources from UIV via UIV Actions NEIs.
 4. OM sends tasks/commands to NE.
 5. OM triggers manual activities to OrderHub (inserts into work queue).
 6. OrderHub user completes manual tasks; informs OM to continue (with data).
 7. OM issues commands to NE/IT systems via NEIs.
 8. OM commits resources in UIV; updates service instance via UIV Actions NEI.
-

What is OrderHub?

Why Use OrderHub?

- Single UI boosts productivity; reduces manual errors; ensures timely/error-free delivery.
- Integrates with Nokia Orchestration Center, FlowOne, third-party systems via adapters (avoids copying info).

- Web form builder for custom forms.
- Defines user groups/roles for access control.
- Deployable in cloud.

What is OrderHub?

- Manages manual orders if no CRM/sales portal.
- Handles manual tasks: Equipment install, fallout, validation, enrichment.
- Monitors/interacts with orders (search, details).
- Features for FlowOne:
 - o Order Entry: Create/submit orders (new/modify/disconnect services).
 - o Search/View: Monitor execution/status; detailed views.
 - o Manage Tasks: Pre-configured queues for enrichment/fallout/validation.
- Separate from other FlowOne components; communicates via FlowOne API (e.g., reads products from Catalog; sends orders to OM).
- No own user management; uses external (e.g., Active Directory, Keycloak).
- User roles define UI access/actions.
- Work Queues: Sub-queues; restricted (linked to work groups) or non-restricted (all users).
- Tasks from OM include parameters for queue and type (defines form/wizard flow).

Summary

- Nokia's system for orders/manual processes.
 - Integrates with third-party via adapters.
 - Relies on external auth solutions.
-

Basic Concepts: Configuration and Tools

InstantLink requires regular maintenance/monitoring for health/performance.

- Tools: Web UI for monitoring; command-line for operations (e.g., config changes, NEMO settings, logs, performance).
- Database stores requests/status and configuration.
- UI Modes:
 - o Production: Safe; monitor flow/performance/config/logs; no changes.

- o Configuration: Allows changes (e.g., params, NE settings, BST logic).
Use only when needed; recommend Production otherwise.
-

Start and Stop IL

- Use `ctl_control` command-line utility.
 - Start: `ctl_control -start` (starts components in order; shows 'Running' status).
 - Stop: `ctl_control -stop`.
 - Reconfig: `ctl_control -reconfig` (after config changes).
 - Status: `ctl_control -status`.
 - Verbose: Add `-v` for more output.
 - Never start components manually; use `ctl_control` for order.
 - Once running, use UI for operations (e.g., config changes, NEMO, logs, performance).
 - System Status Page: Compact (overall); Full (component statuses, uptime, logs).
-

Logs

- Written to filesystem; some available in UI.
 - View all via command-line.
 - Key Directories:
 - o `~/sas/logdir/logs`: System component logs.
 - o `~/sas/logdir/ne_logs`: NEI logs.
 - Other: Northbound interfaces/custom modules (check docs for locations).
 - Load Distribution Page: Shows real-time request processing; identifies bottlenecks.
 - Import/Export: UI (export in Production; import in Configuration);
command-line utility (details in Operation and Maintenance Guide).
-

File System and Database Maintenance

InstantLink generates data in DB/filesystem; affects performance if unmanaged.

Database Maintenance

- DB health critical; updates multiple times per request/task.
- If full/slow, platform fails/stops.
- Cleanup Methods (never run together):
 - o **Automatic:** Enable GRC param "Clean old requests from the database automatically" (disabled default). Set hours to store successful vs. failed/canceled/partly failed requests. Effective immediately; heavy—use for low-load (test/dev) envs.
 - o **Manual/Scheduled:** `dbcleaner.ksh` (command-line; run as cron during off-peak; heavy operation).
- Cleanup only acknowledged responses; use `ack_requests.ksh` to auto-acknowledge old unacknowledged requests (e.g., `ack_requests.ksh 7` for >7 days).
- Monitoring: `dbmonitor.ksh` (run background; alarms on low tablespace; visible in UI; integrable with external systems).
- Analysis: `il_db_analyse` (run regularly; light; even during peak; optimizes performance; avoid on empty DB).

File System Maintenance (Logs/Files)

- Logs grow continuously; manage to free space.
- **NEI Logs:**
 - o Split daily at midnight: `split_file.ksh /path/to/ne_logs` (new logs start; old to 'old' dir with timestamp; cron job).
 - o Archive: `archive_ne_logs.ksh <days>` (e.g., >X days to archive dir; after split).
 - o Delete: `delete_archived_ne_logs.ksh <days>` (e.g., >60 days from archive).
- **Other Logs/Files:** Use `archive_file.ksh` (e.g., archive >5 days *.log to backup) and `delete_file.ksh` (e.g., delete *.xml from archive).
- Scheduling: Use Unix Crontab (e.g., DB analysis at 22:30 daily; NE logs split at 00:00).
 - o Example Crontab:

```
30 22 * * * /home/ilink/sas/bin/il_db_analyse
0 0 * * * /home/ilink/sas/bin/split_file.ksh
/home/ilink/sas/logdir/ne_logs
```

Performance Tuning

Objectives

- Configure RPE, SME, TE for optimal performance.

Request Processing Engine Configuration (RPE)

- One per InstantLink instance.
- Memory: Set in `start_RequestProcessingEngine.ksh` (MAX_HEAP var; default 512MB; production: ≥ 2 GB; restart required).
- GRC Parameters:
 - o `exec_list_size`: Max requests before delaying (braking mechanism: 70% = 0.4s delay; 80% = 1s; 90% = 2s; exceed = dump to DB). Set to peak load; ensure RPE memory sufficient. Check braking via Load Distribution Page.
 - o `max_active_count`: Max requests passed to SME before braking.

Service Module Engine Configuration (SME)

- Memory: Set in `start_ServiceModuleEngine.ksh` (MAX_HEAP; restart required; depends on simultaneous requests/workflow complexity; ensure host memory available).
- Default: 2 SMEs.
- Add more in production (e.g., 8) for redundancy (distributes load; losing one not catastrophic).

Task Engine Configuration (TE)

- One per instance.
 - Memory: Set in `start_TaskEngine.ksh` (MAX_HEAP; restart required).
 - Consumption: Depends on Network Model (concurrent sessions launch Macroserver/NEI instances); task size; NEI types (Java > Expect).
 - If out of memory (no spare physical): Adjust Network Model to limit sessions.
-

NEI Development

NEI Architecture Refresher

- **Macro Servers**: Java or Expect; handle communication with InstantLink; provide base services for NEIs.

- **NEIs:** Plugins extending Macro Server for specific external systems/NE; handle NE communication and command execution.
- **Task Management by TE:**
 - a. Task arrives; TE starts relevant Macro Server.
 - b. TE sends task data to Macro Server.
 - c. Macro Server starts NEI; passes data.
 - d. NEI executes in NE.
 - e. Queue empty: TE terminates Macro Server; NEI logs out/terminates connection.
- **Log Files:**
 - o IO Log: TE ↔ Macro Server messaging (task/response params).
 - o ERR Log: Macro Server/NEI internals (errors/debug).
 - o MML Log: NEI ↔ NE messaging (commands/responses).

Basic Principles of Developing NEIs

- **Process Guide:**
 - a. Design architecture diagram (InstantLink ↔ NE communication).
 - b. High-level functionality picture; detailed flowcharts (e.g., create/modify subscriber).
 - c. Write Functional Description: Detail tasks, mandatory/optional params.
 - d. Specify utilities/classes/methods/procedures (e.g., param reading, error handling; cover all errors/successes).
- **Development Steps:**
 - a. Design/verify params/commands against vendor docs.
 - b. Document: Functional Description; code comments (explain non-obvious, decisions, options).
 - c. Implement basic functions; unit test (simulator/real system; no InstantLink).
 - d. Implement specials (e.g., phase messages, error handling).
 - e. Test Plan: State required functionality; integration test in dev env (send requests); create test report.
 - f. Package/deploy to production.

Parameter and Extension Files

- Configured via Global Resource Configuration (GRC) file (.mds.rc; pointed by env var RCFILE).

- Each Macro Server has GRC section for behavior params.
- For NEI behavior changes without code: Add NEI section in GRC (e.g., SOAP endpoint).